

Estratégias de Busca com Informação

Busca heurística



Gabriel P. Oliveira

Inteligência Artificial
2026/1

Problem-solving agents

Um tipo de agente que decide o que fazer procurando **sequências de ações que o leva a um objetivo** (estado desejado) para um dado problema

Formulação do Problema

1. **Estado inicial** em que o agente começa
2. **Conjunto de ações** disponíveis para o agente
3. **Modelo de transição** (resultados das ações)
 - Função sucessora (pares <ação, estado>)
 - Estado inicial, ações e modelo de transição definem o **espaço de estados do problema**, ou seja todos os estados atingíveis a partir do estado inicial (grafo)
4. **Teste do objetivo**
 - Estado(s) explícito(s) ou propriedade abstrata (e.g., xeque-mate)
5. **Custo da solução** (do caminho)
 - Custo do caminho é a soma do custo dos passos individuais
 - Solução ótima é aquela cujo caminho tem o menor custo

Tipos de Busca

1. Busca sem informação

- Não têm nenhuma informação adicional sobre estados além daquelas fornecidas na definição do problema

2. Busca com informação (heurística)

- Sabem se um estado não objetivo é mais “promissor”

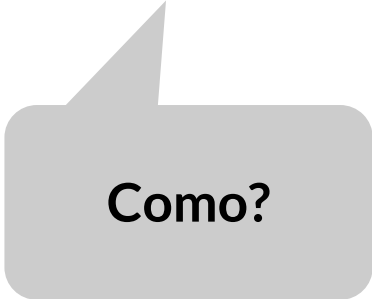
Tipos de Busca

1. Busca sem informação

- Não têm nenhuma informação adicional sobre estados além daquelas fornecidas na definição do problema

2. Busca com informação (heurística)

- Sabem se um estado não objetivo é mais “promissor”



Como?

Busca com Informação

Ideia → Se houver algum conhecimento maior sobre o problema que possa ser utilizado, a busca pode ser muito mais eficiente

Função de Avaliação $f(n)$

- ▷ O nó a ser expandido é escolhido de acordo com uma função de avaliação
 - Expande-se o nó com menor $f(n)$
- ▷ Em geral, $f(n)$ utiliza alguma heurística

Busca com Informação

Qual a diferença entre usar um algoritmo de busca com informação e a busca com custo uniforme (Dijkstra)?

Busca com Informação

Qual a diferença entre usar um algoritmo de busca com informação e a busca com custo uniforme (Dijkstra)?

- ▷ **Dijkstra:** escolhe sempre o caminho com menor custo acumulado até agora
- ▷ **Com informação:** escolhe com base em:
custo já percorrido + estimativa do custo até o objetivo

A execução com fila de prioridades (heap) é a mesma

Busca com Informação

Qual a diferença entre usar um algoritmo de busca com informação e a busca com custo uniforme (Dijkstra)?

- ▷ **Dijkstra:** escolhe sempre o caminho com menor custo acumulado até agora
- ▷ **Com informação:** escolhe com base em:
custo já percorrido + **estimativa do custo até o objetivo**

Função heurística

A execução com fila de prioridades (heap) é a mesma

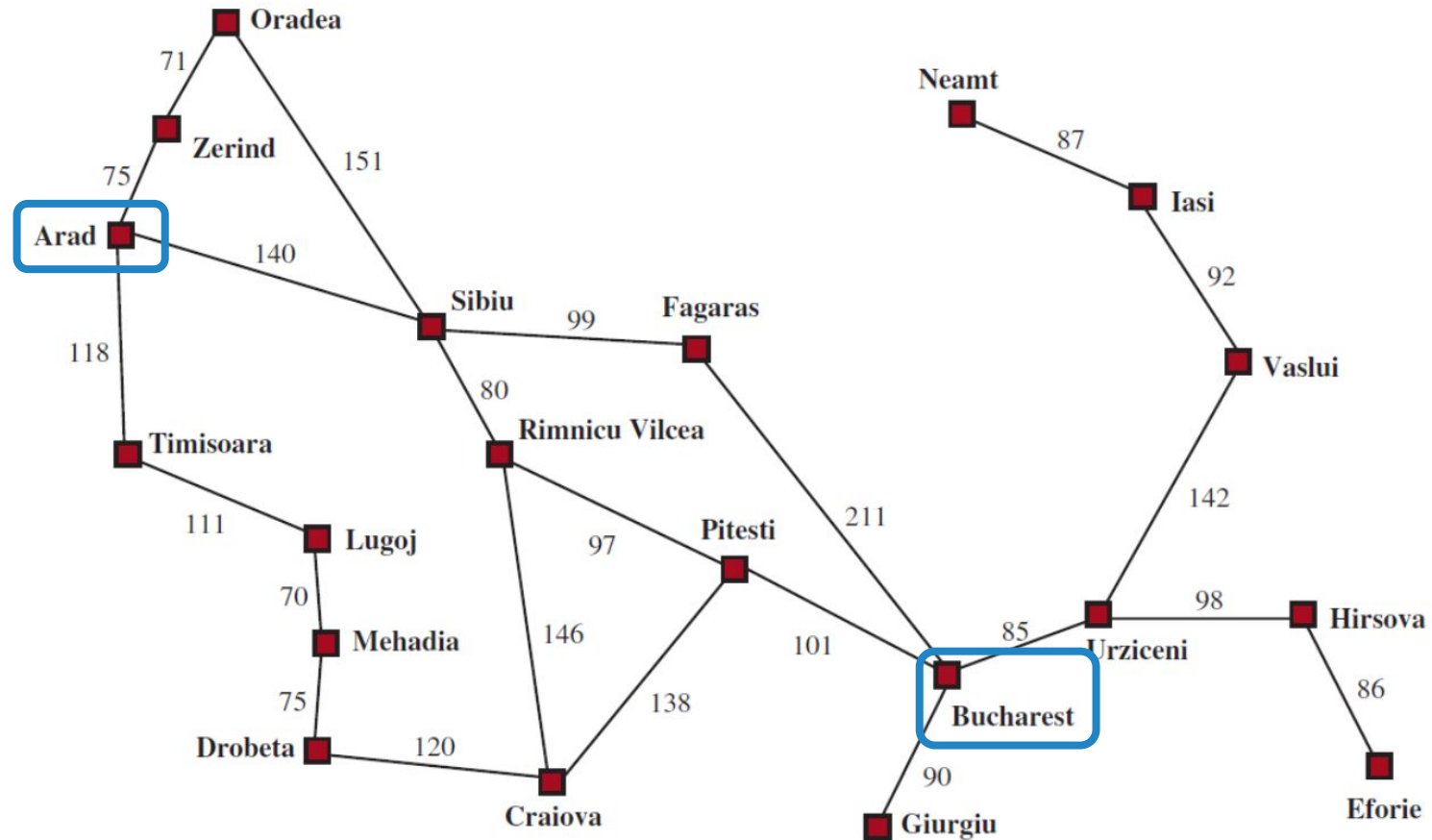
Função Heurística

Intuição → Fornece uma indicação do melhor caminho até a solução desejada

- ▷ $h(n)$ é o custo **estimado** do melhor caminho de um nó n até o objetivo
 - Se n for objetivo, $h(n) = 0$

Função Heurística

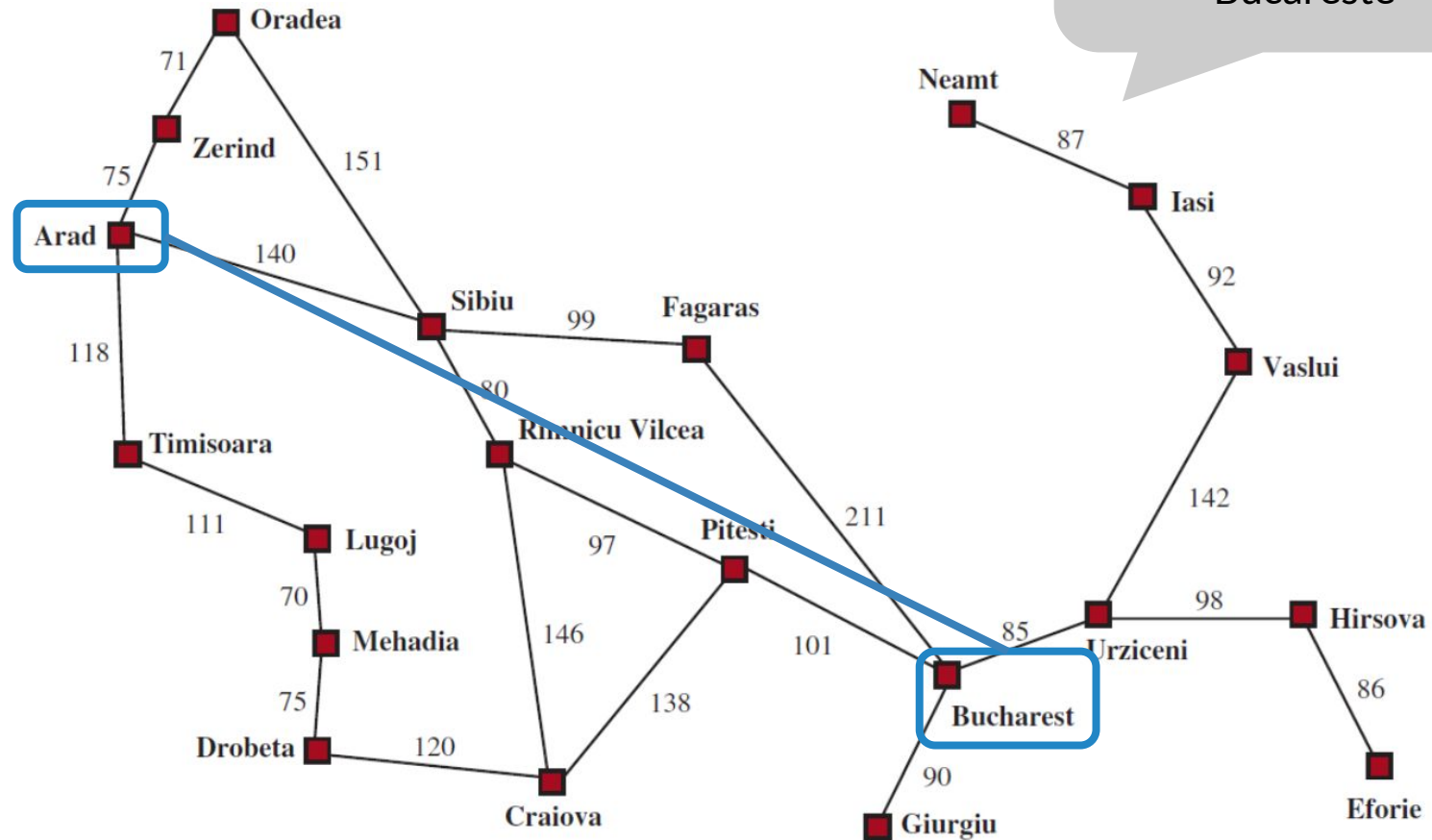
Viagem na Romênia: Arad → Bucareste



Função Heurística

Viagem na Romênia: Arad → Bucareste

A função heurística $h(n)$ pode ser a **distância em linha reta** entre n e Bucareste



1.

Busca Gulosa

Uma solução simples usando heurísticas

Busca Gulosa (*Greedy*)

Ideia → Expandir o nó que está mais próximo do objetivo de acordo com a função heurística

▷ $f(n) = h(n)$

Por que “gulosa”?

- ▷ O método procura **reduzir o custo imediato** para alcançar o objetivo na expansão de cada nó, porém sem se preocupar com o custo total do caminho

Busca Gulosa

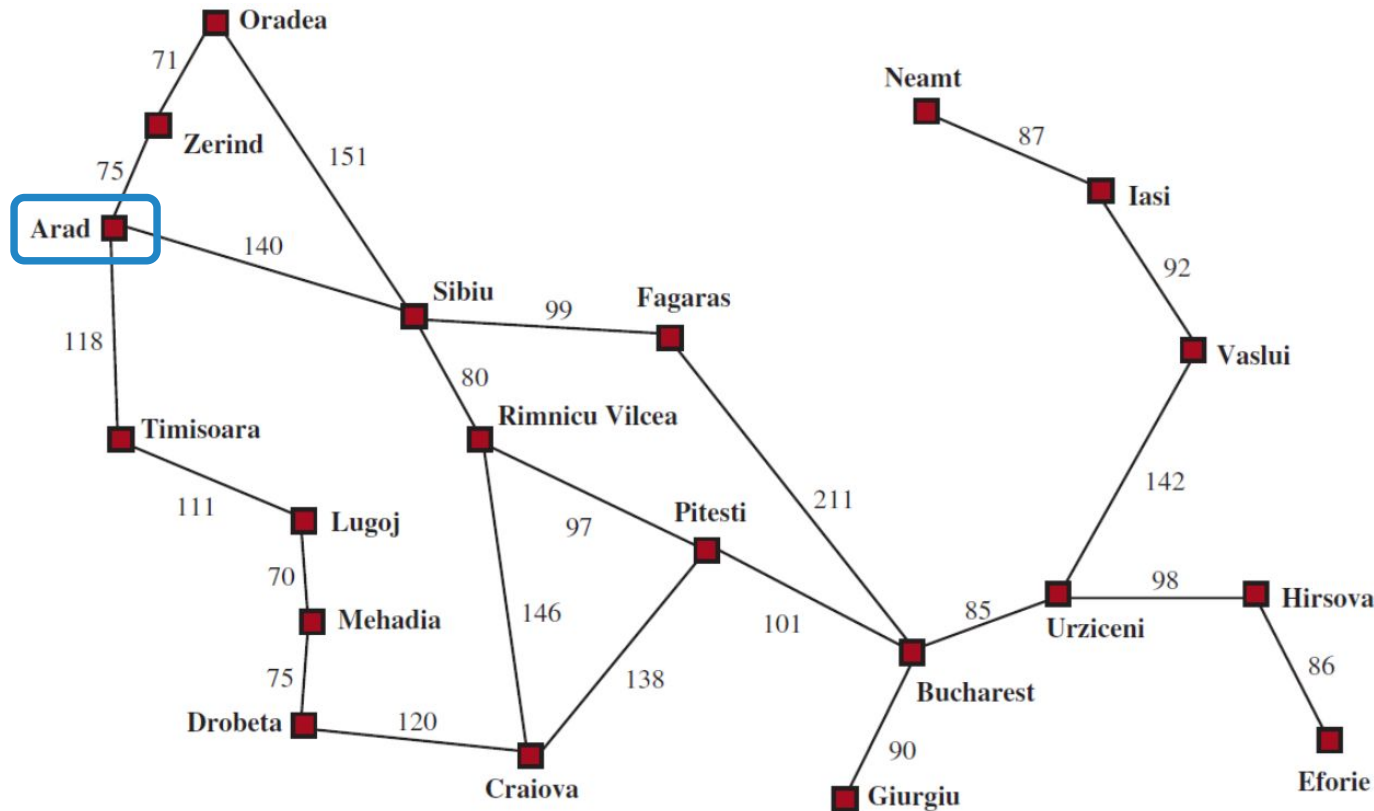
Viagem na Romênia: Arad → Bucareste

Objetivo → Achar um caminho (o melhor) entre Arad e Bucareste

Heurística → distância em linha reta entre as cidades

Busca Gulosa

Viagem na Romênia: Arad → Bucareste



Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Busca Gulosa

Viagem na Romênia: Arad → Bucareste

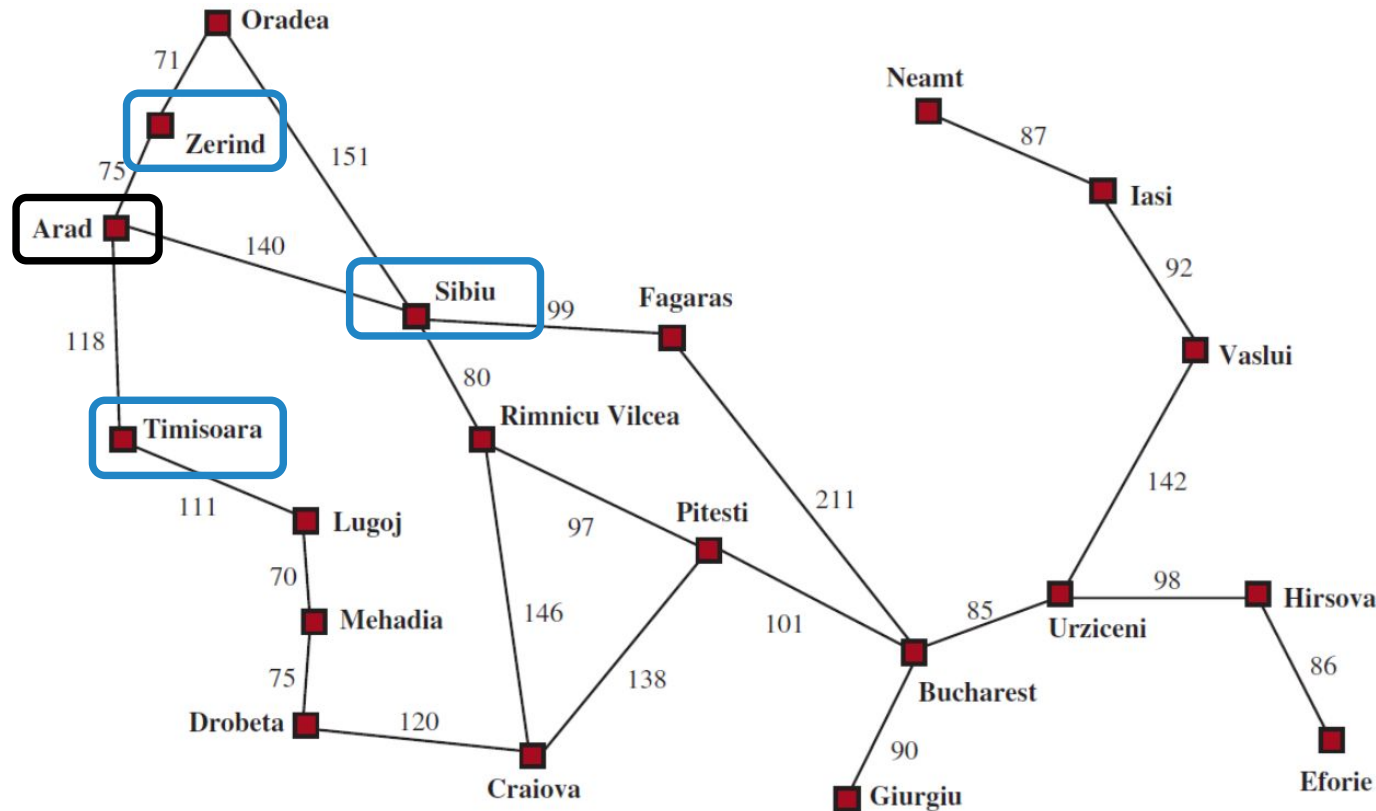
(a) The initial state



Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Busca Gulosa

Viagem na Romênia: Arad → Bucareste



Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

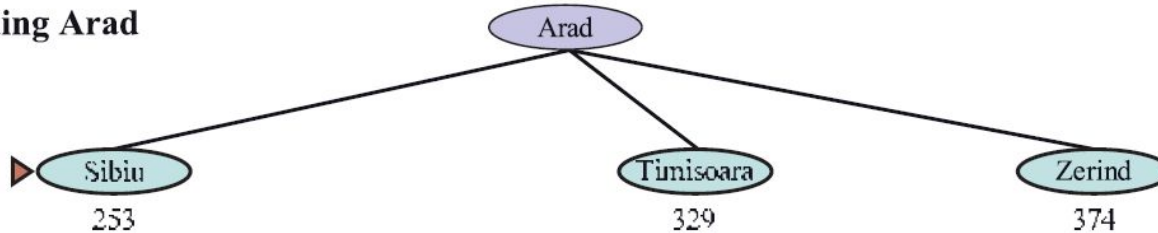
Busca Gulosa

Viagem na Romênia: Arad → Bucareste

(a) The initial state



(b) After expanding Arad

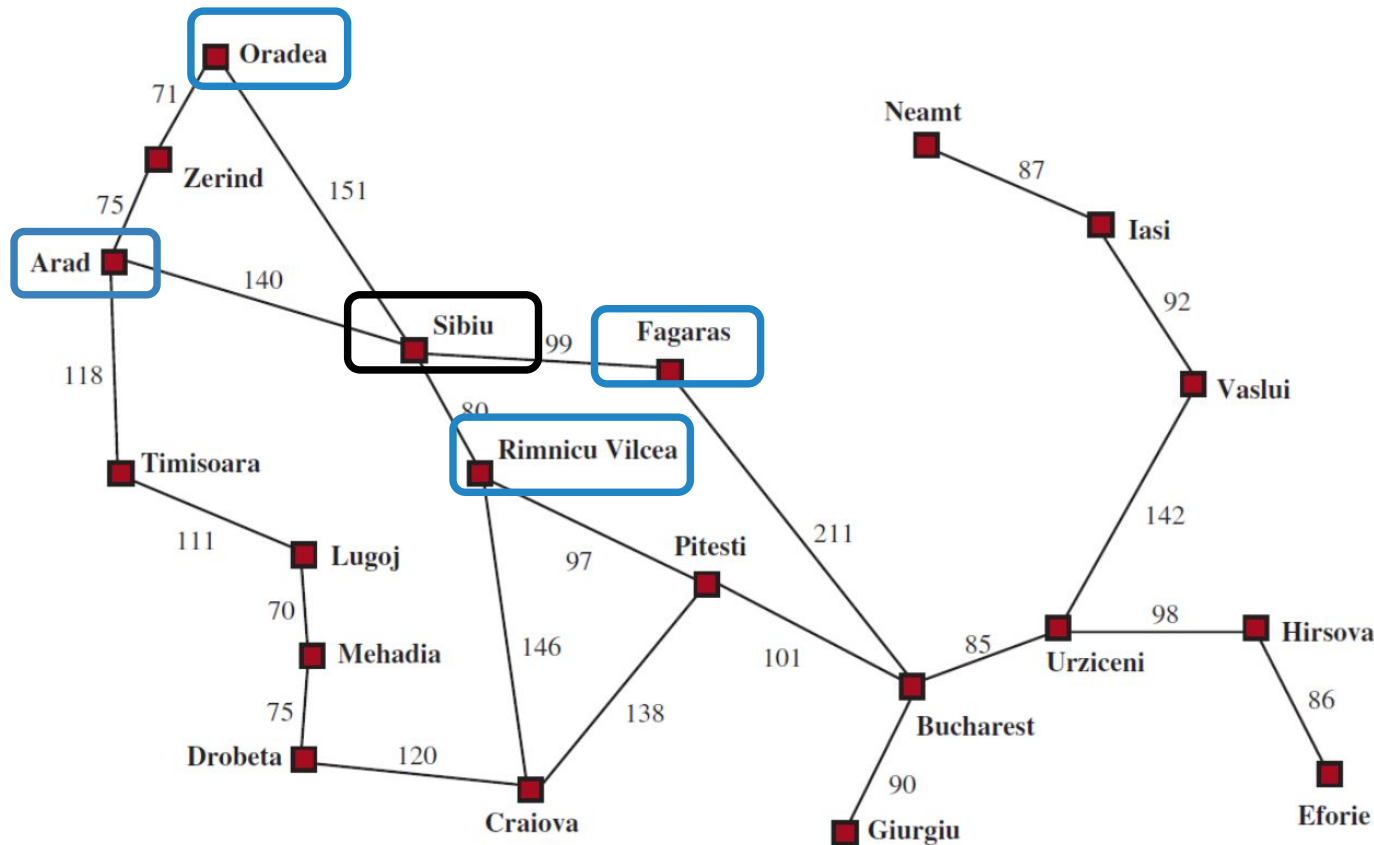


Dist. Linha Reta

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Busca Gulosa

Viagem na Romênia: Arad → Bucareste



Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

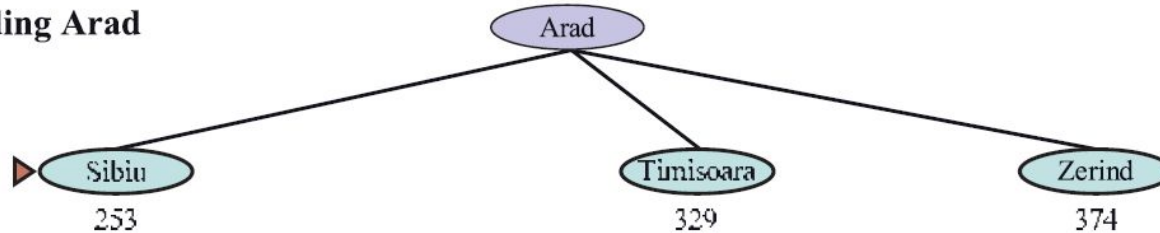
Busca Gulosa

Viagem na Romênia: Arad → Bucureste

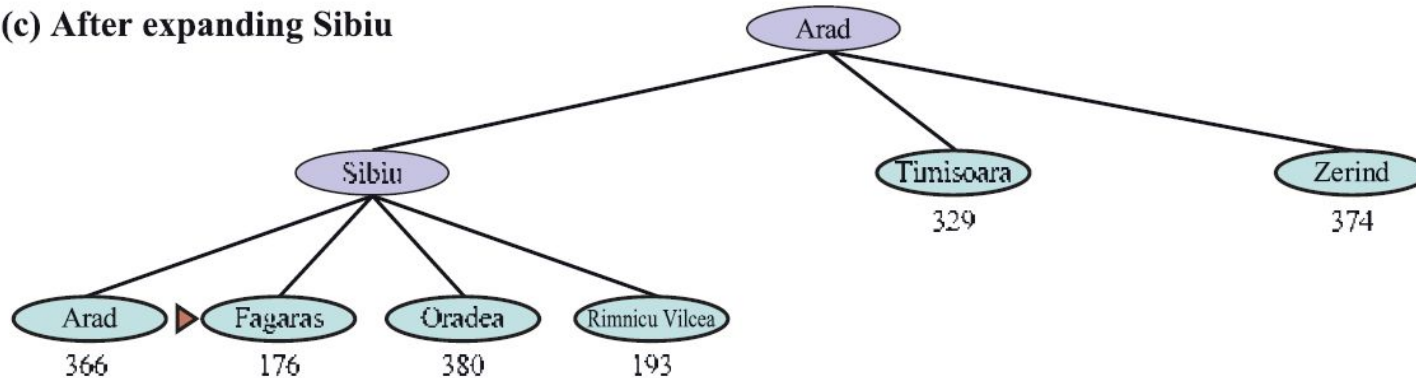
(a) The initial state



(b) After expanding Arad



(c) After expanding Sibiu

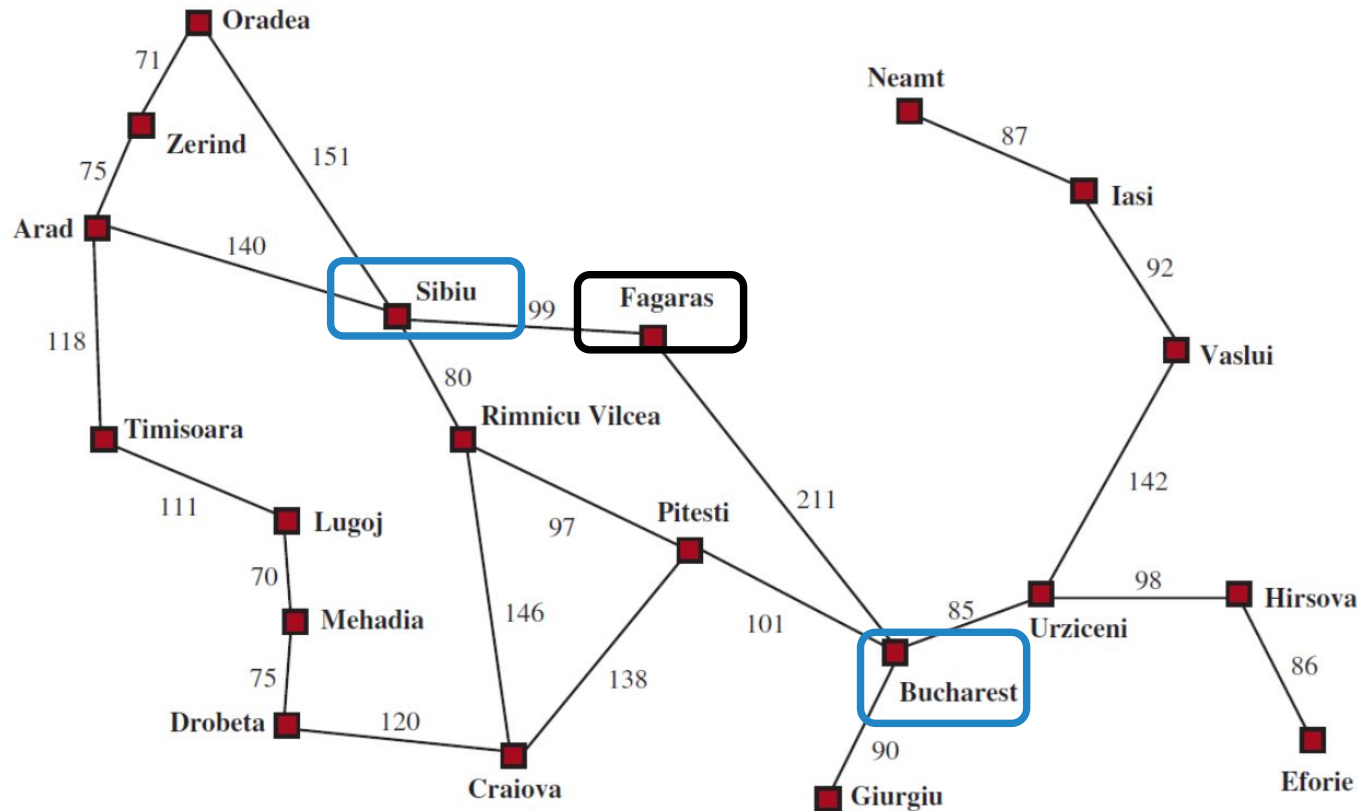


Dist. Linha Reta

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Busca Gulosa

Viagem na Romênia: Arad → Bucareste

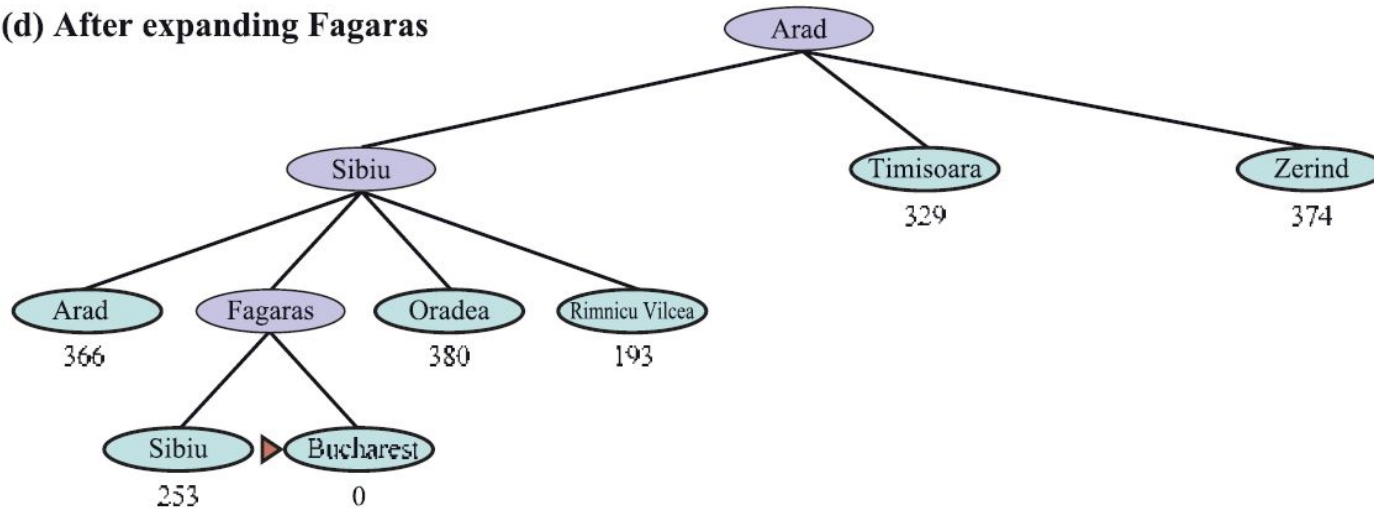


Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Busca Gulosa

Viagem na Romênia: Arad → Bucureste

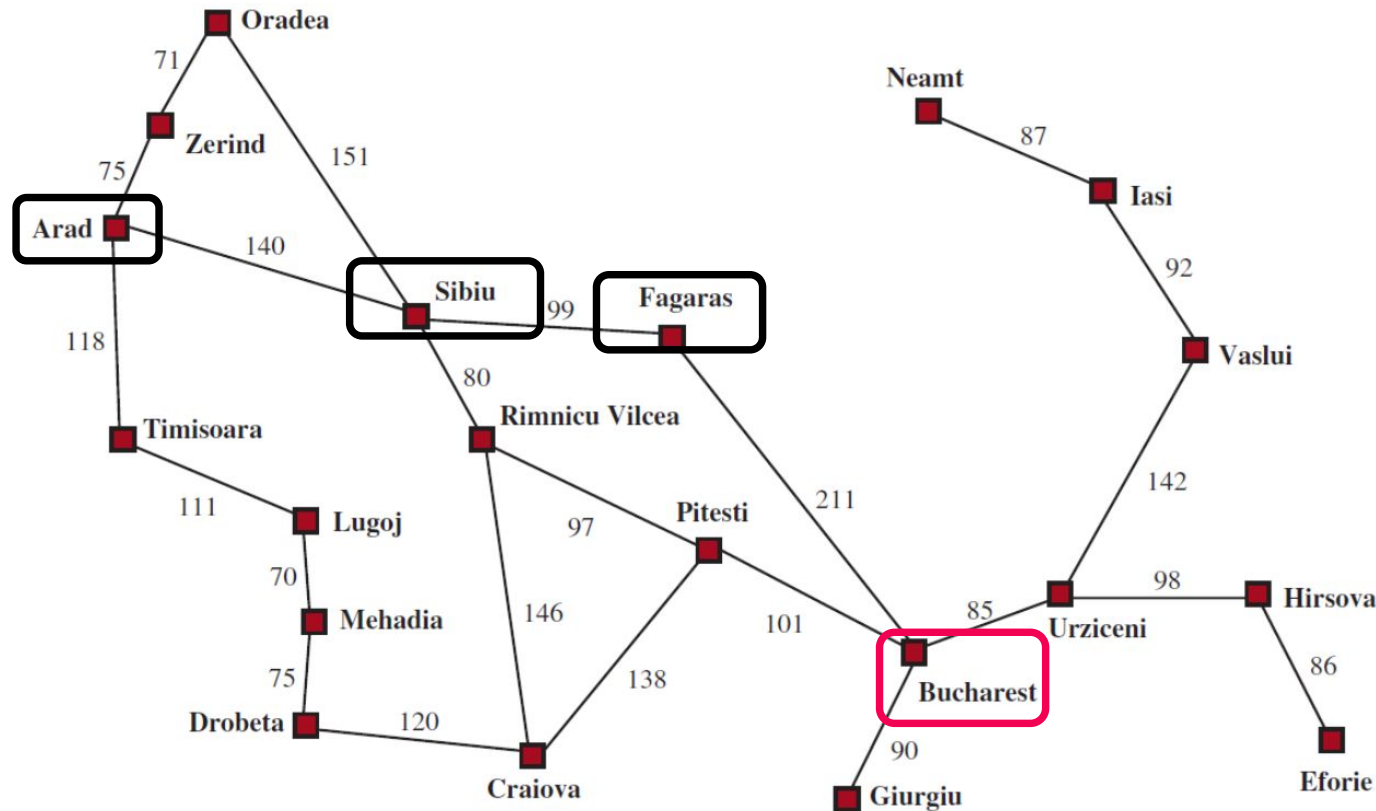
(d) After expanding Fagaras



Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Busca Gulosa

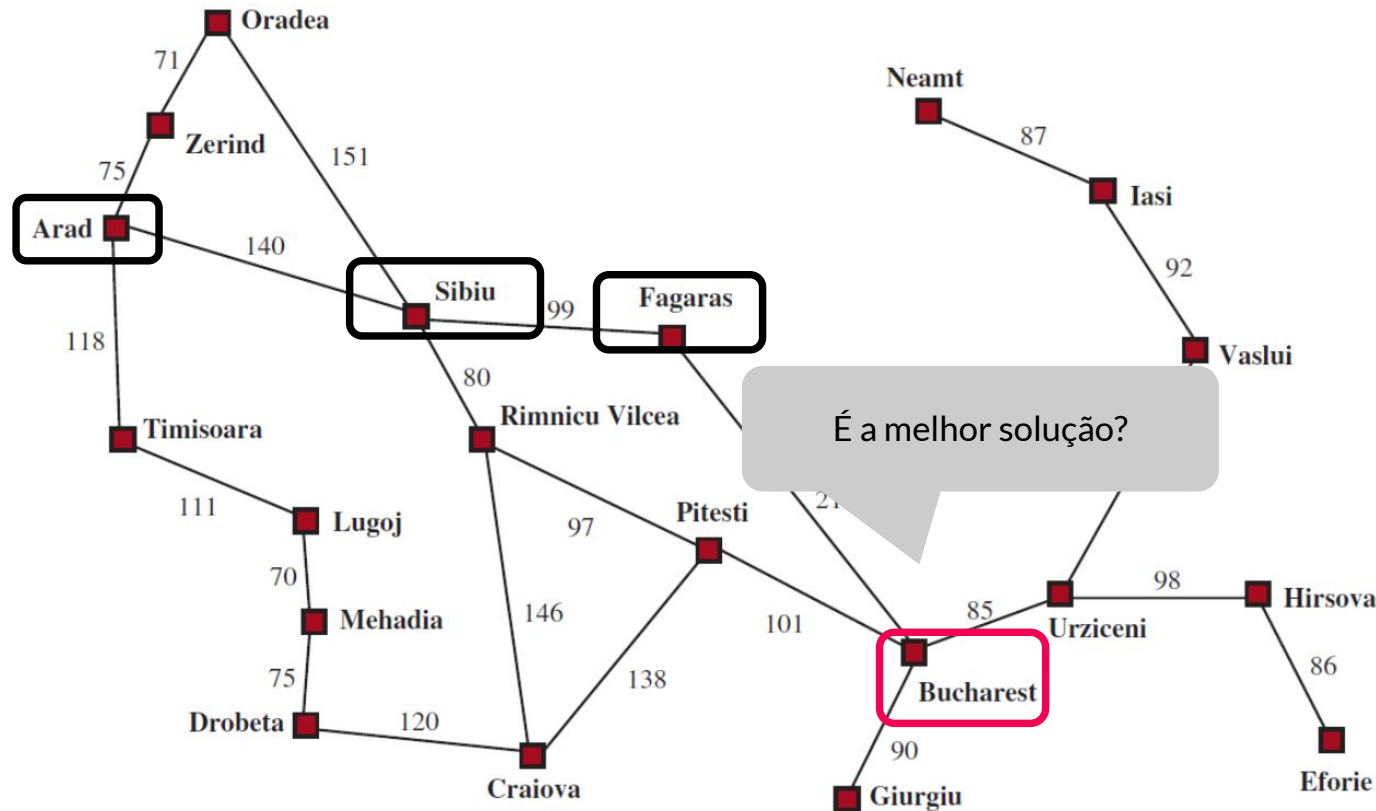
Viagem na Romênia: Arad → Bucareste



Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Busca Gulosa

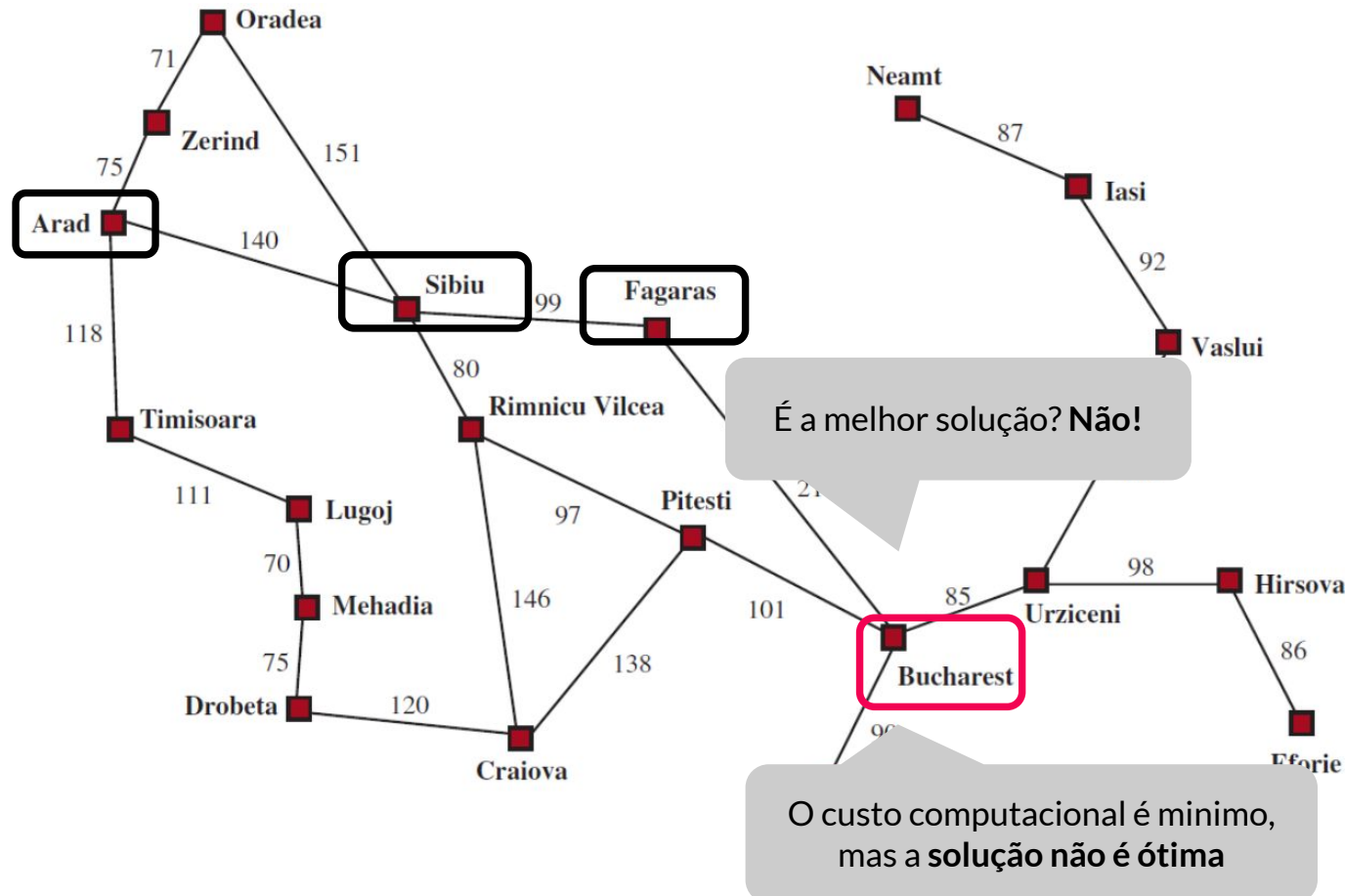
Viagem na Romênia: Arad → Bucareste



Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Busca Gulosa

Viagem na Romênia: Arad → Bucareste



Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Busca Gulosa

Avaliação do algoritmo

- ▷ **Completo:** não (pode entrar em loop)
- ▷ **Ótimo:** não
- ▷ **Complexidade**
 - Considerando branch factor b , profundidade máxima da árvore m
 - Raiz gera b nós no primeiro nível, cada um desses outros gera outros b outros nós, totalizando b^2 no segundo nível, e assim sucessivamente
 - Tempo e espaço: $O(b^m)$

Busca Gulosa

Avaliação do algoritmo

- ▷ **Completo:** não (pode entrar em loop)
- ▷ **Ótimo:** não
- ▷ **Complexidade**
 - Considerando branch factor b , profundidade máxima da árvore m
 - Raiz gera b nós no primeiro nível, cada um desses outros gera outros b outros nós, totalizando b^2 no segundo nível, e assim sucessivamente
 - Tempo e espaço: $O(b^m)$

IMPORTANTE: a escolha da função heurística é fundamental para a eficiência do algoritmo
(heurística boa → execução rápida)

2.

Algoritmo A*

Minimização do custo total estimado da solução

Algoritmo A*

Ideia → Expandir o nó de acordo com a função:

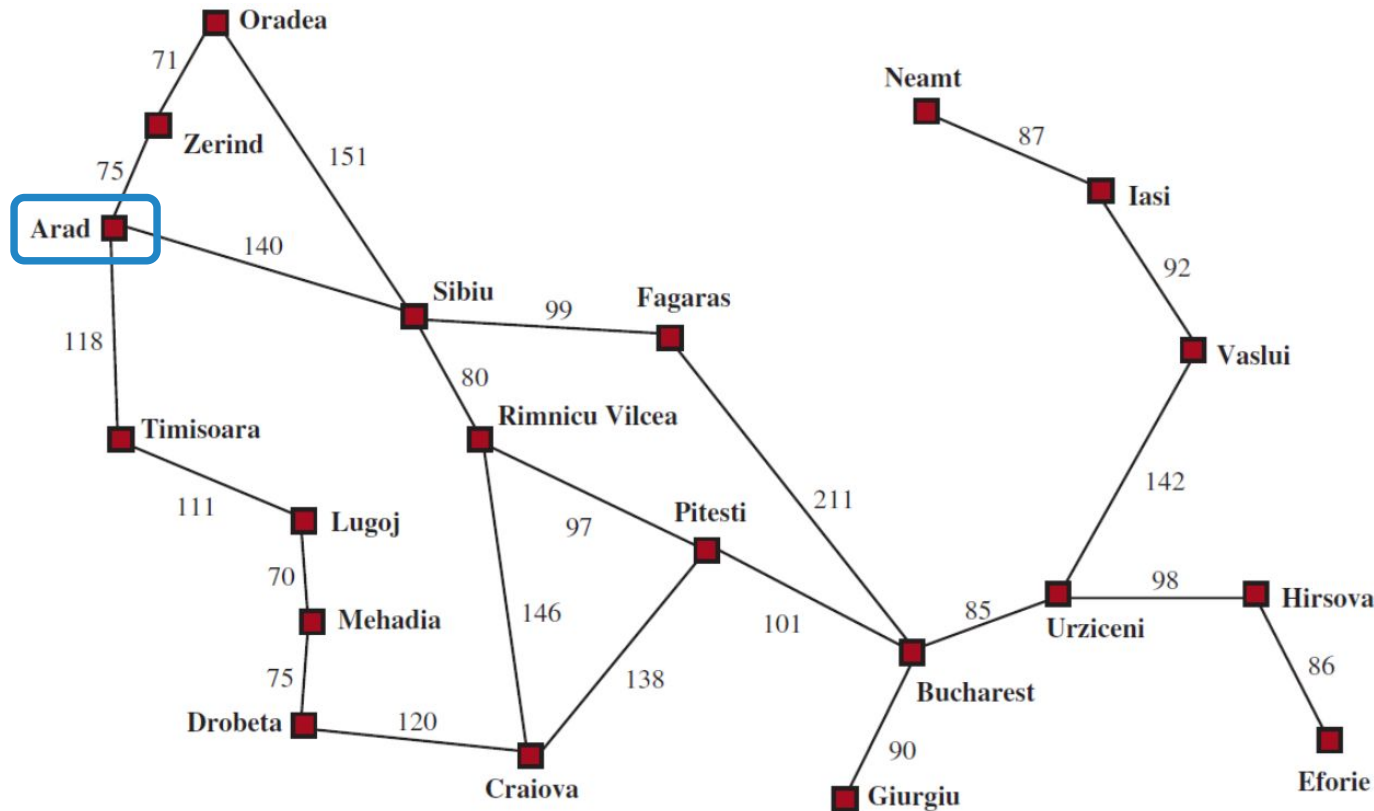
$$f(n) = g(n) + h(n)$$

- ▷ $g(n)$: **custo real** de chegar ao nó atual
- ▷ $h(n)$: **estimativa do custo** do nó atual para o objetivo

$f(n)$ é então a estimativa do custo da melhor solução que passa por n

Algoritmo A*

Viagem na Romênia: Arad → Bucareste

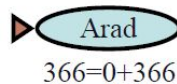


Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Algoritmo A*

Viagem na Romênia: Arad → Bucareste

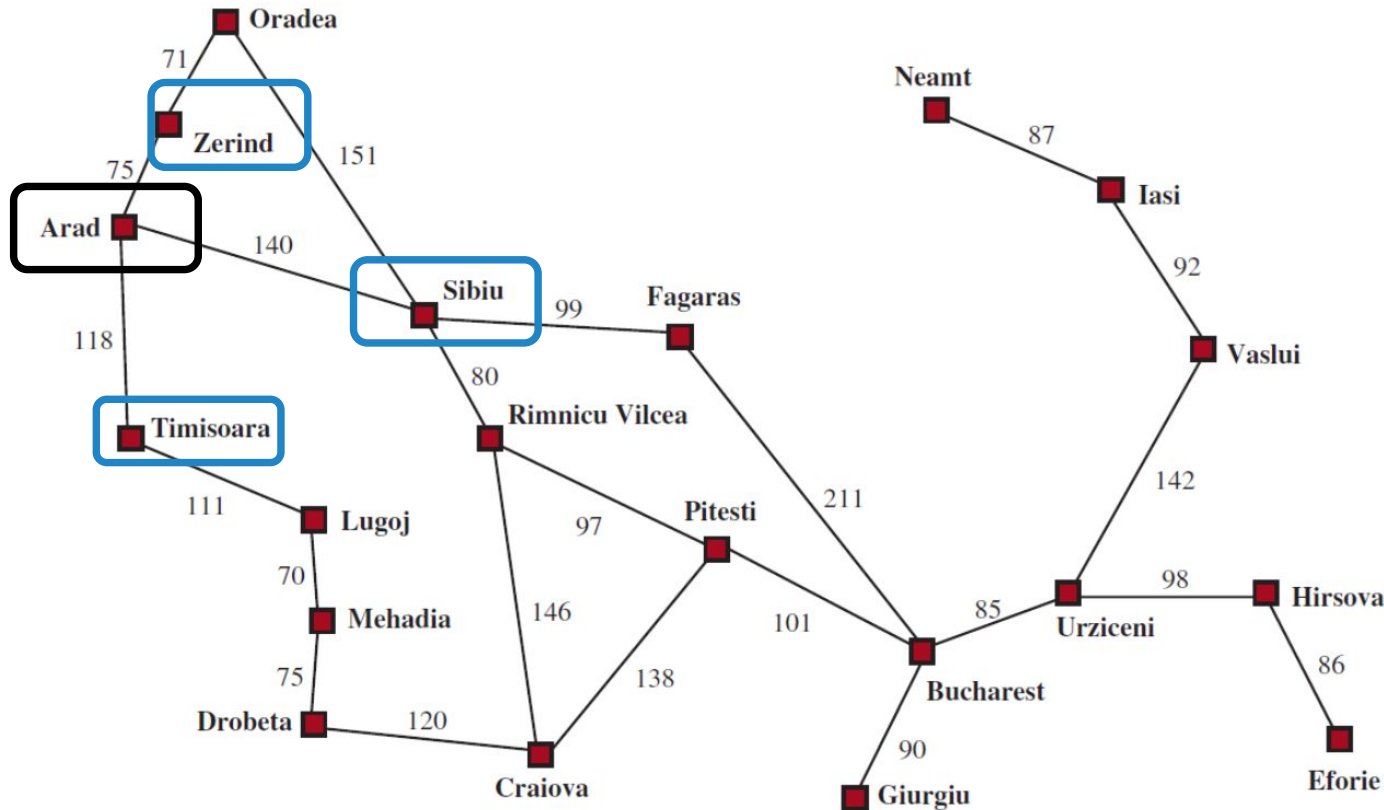
(a) The initial state



Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Algoritmo A*

Viagem na Romênia: Arad → Bucareste

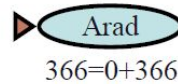


Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

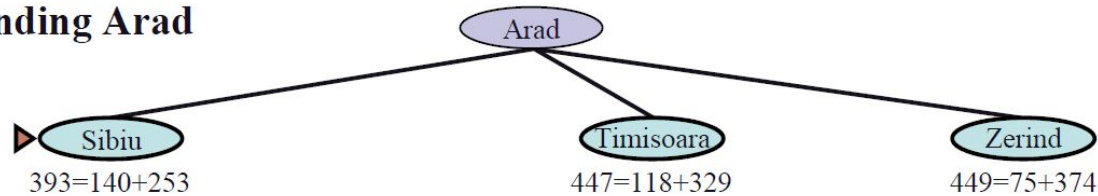
Algoritmo A*

Viagem na Romênia: Arad → Bucureste

(a) The initial state



(b) After expanding Arad



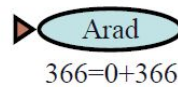
Custo total = custo até aqui + dist. linha reta (heurística)

Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

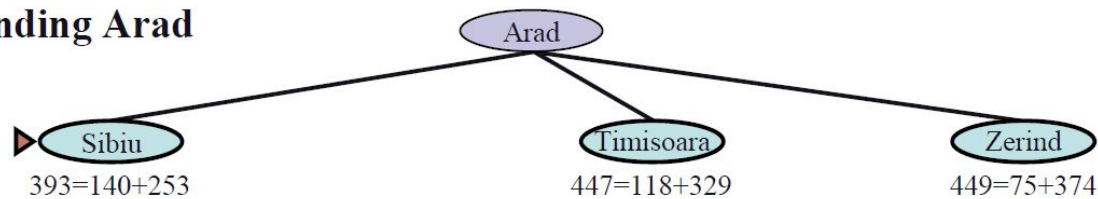
Algoritmo A*

Viagem na Romênia: Arad → Bucareste

(a) The initial state



(b) After expanding Arad

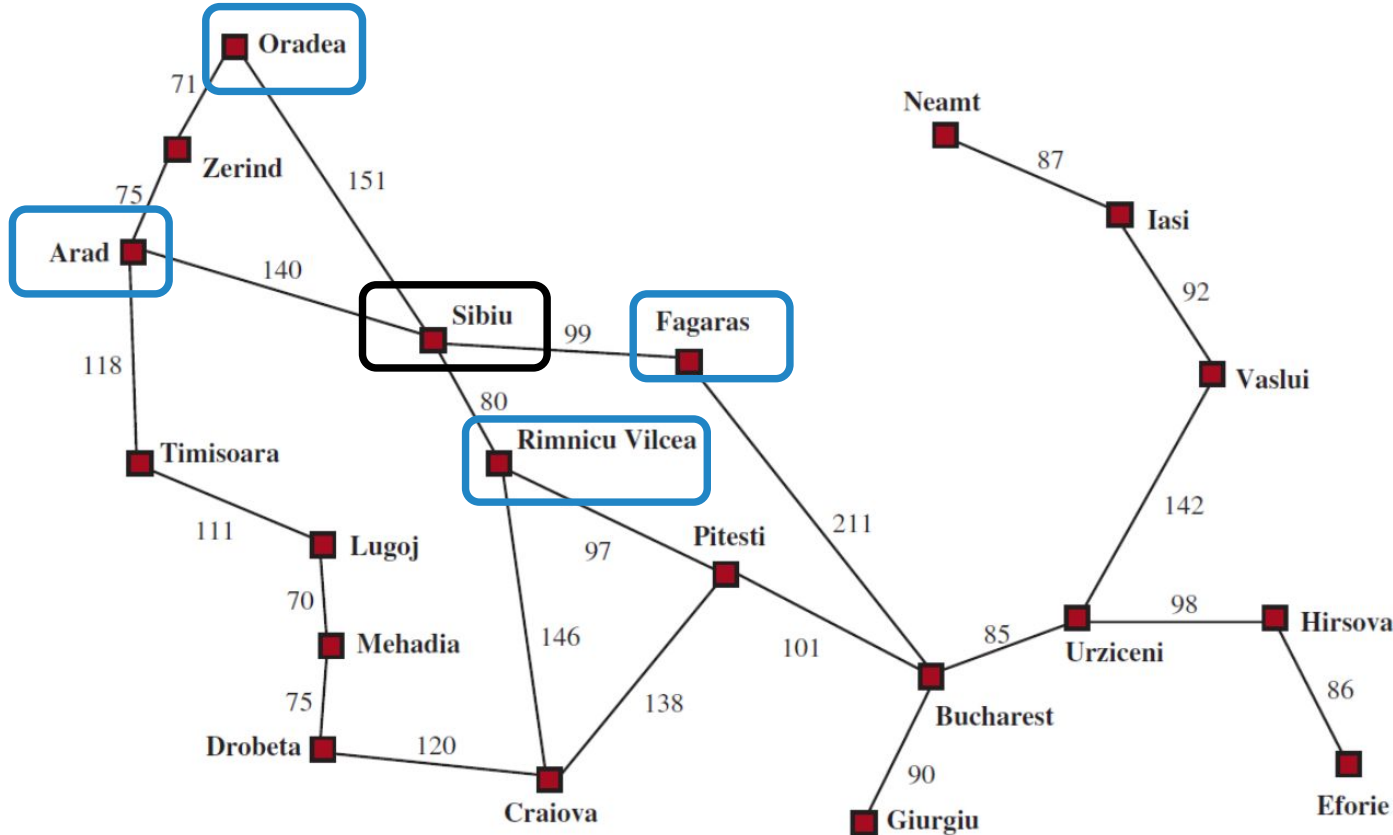


O nó a ser expandido é o com menor custo

Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Algoritmo A*

Viagem na Romênia: Arad → Bucareste

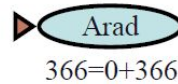


Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

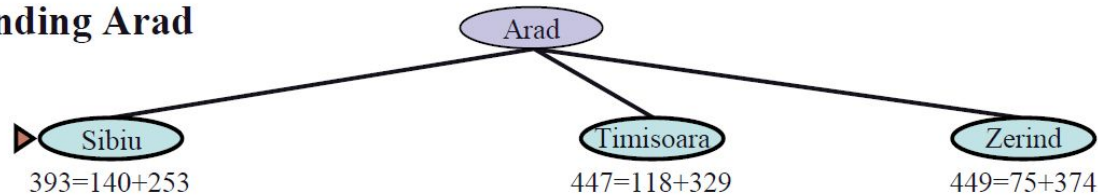
Algoritmo A*

Viagem na Romênia: Arad → Bucureste

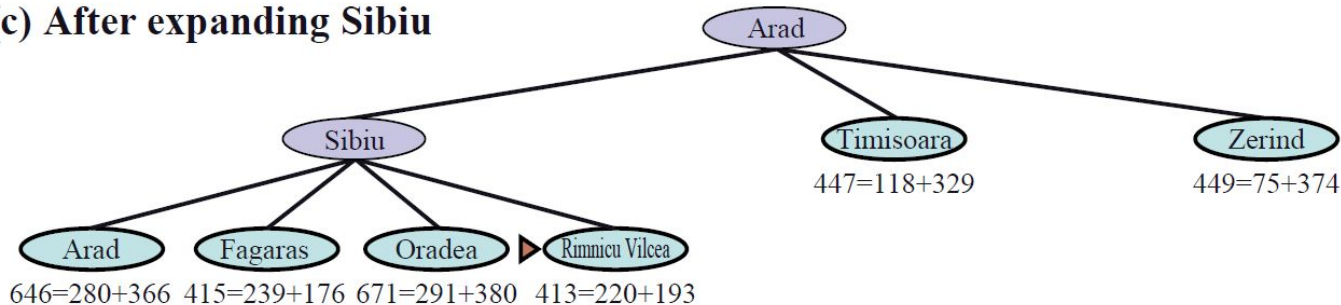
(a) The initial state



(b) After expanding Arad



(c) After expanding Sibiu

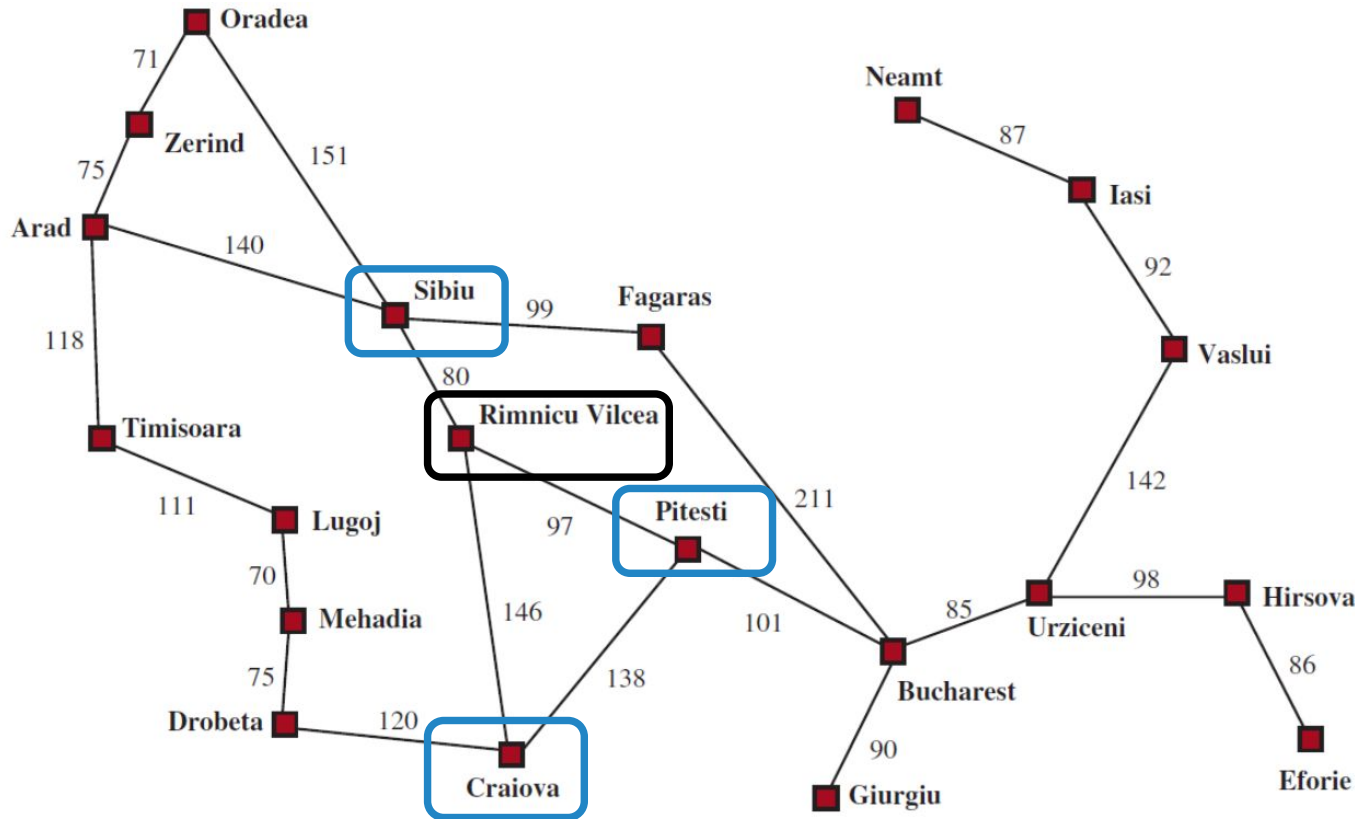


Dist. Linha Reta

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Algoritmo A*

Viagem na Romênia: Arad → Bucareste

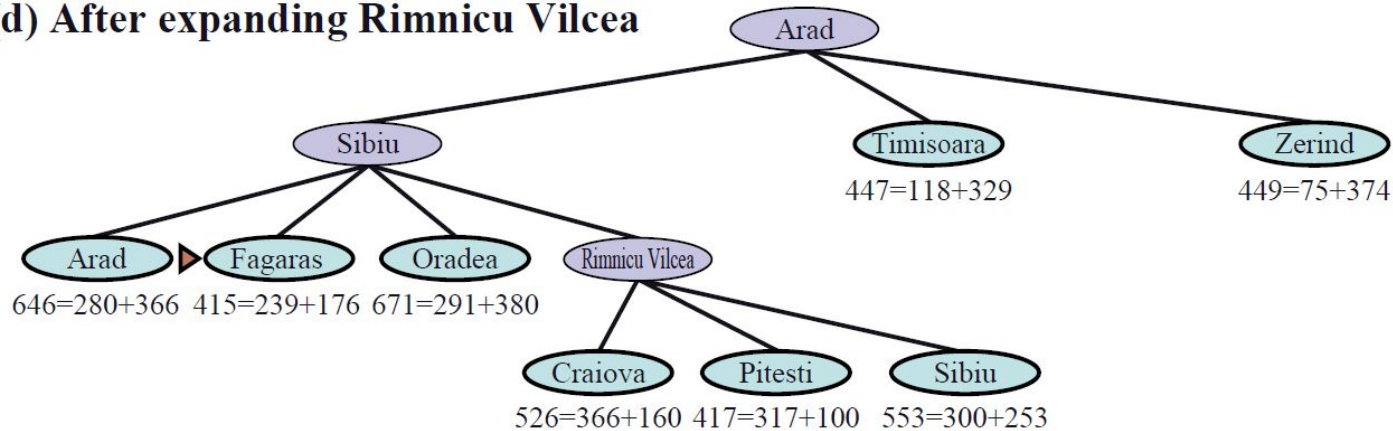


Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Algoritmo A*

Viagem na Romênia: Arad → Bucareste

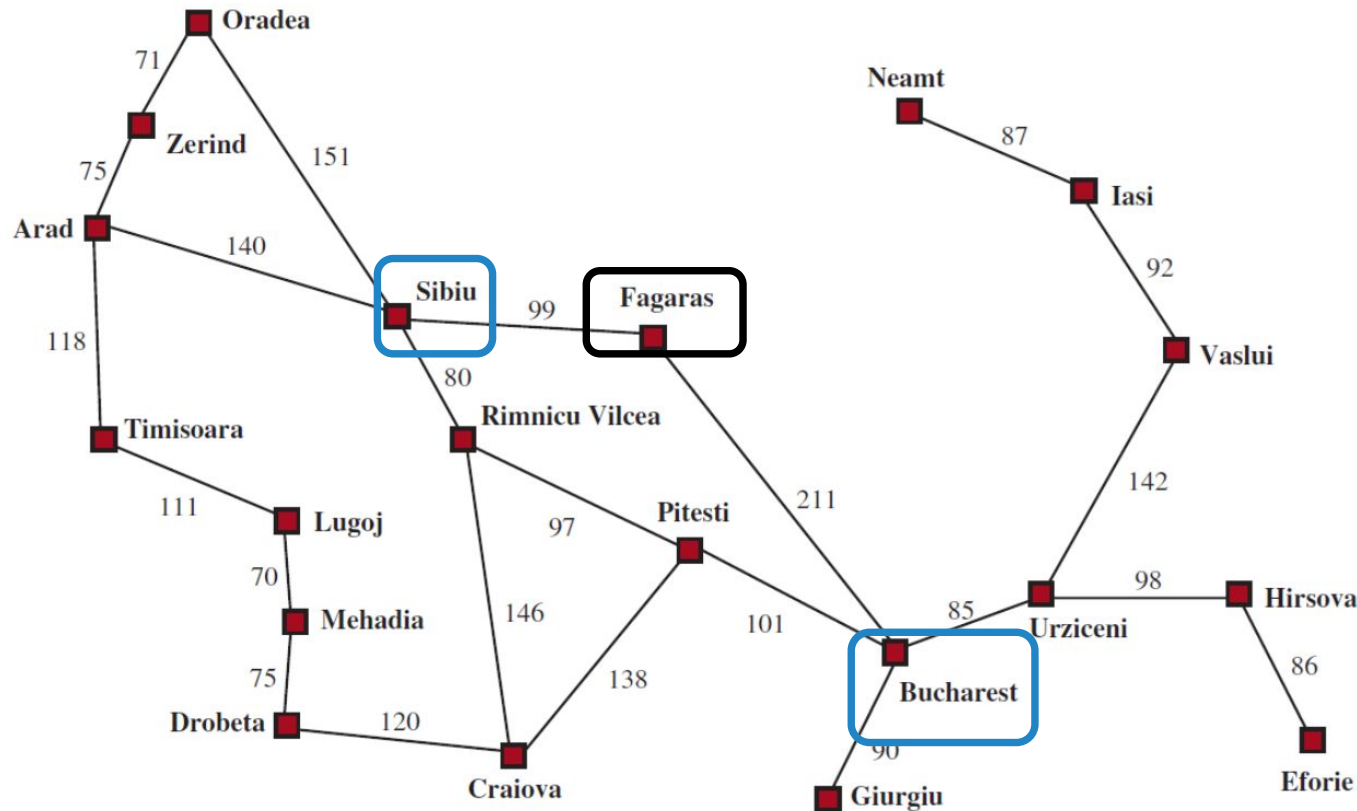
(d) After expanding Rimnicu Vilcea



Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Algoritmo A*

Viagem na Romênia: Arad → Bucareste

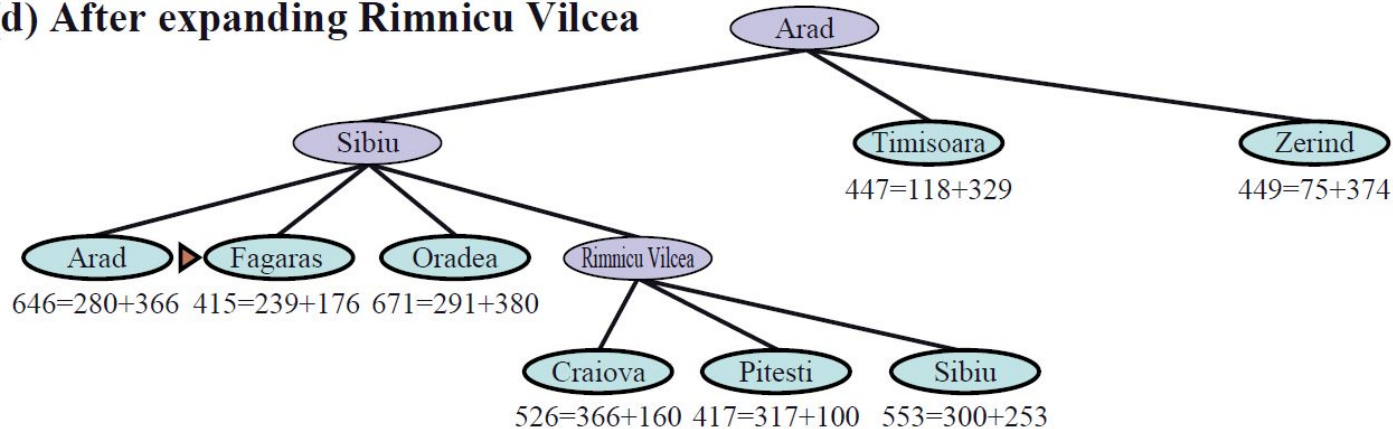


Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

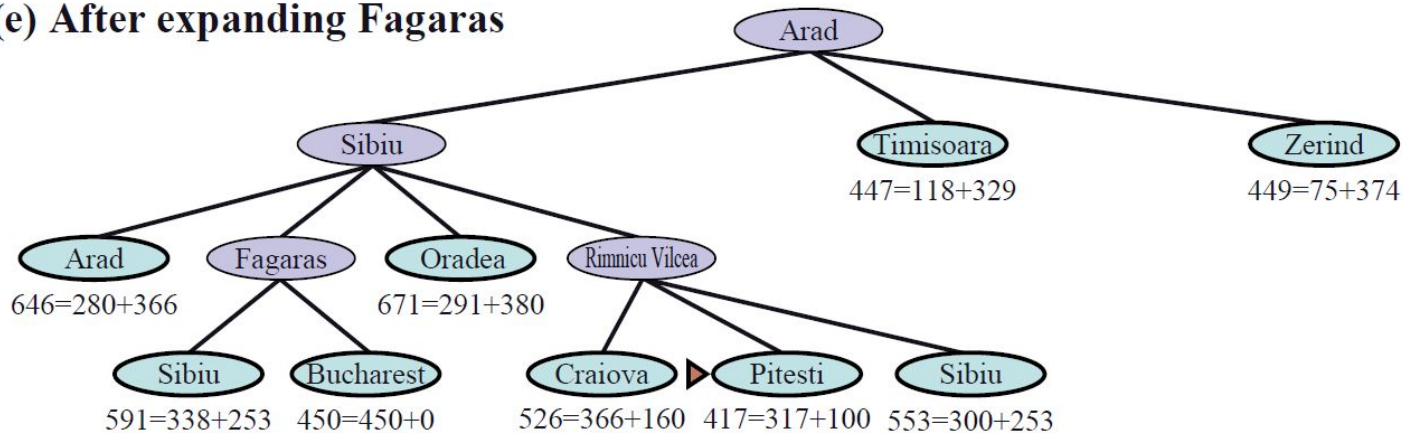
Algoritmo A*

Viagem na Romênia: Arad → Bucareste

(d) After expanding Rimnicu Vilcea



(e) After expanding Fagaras



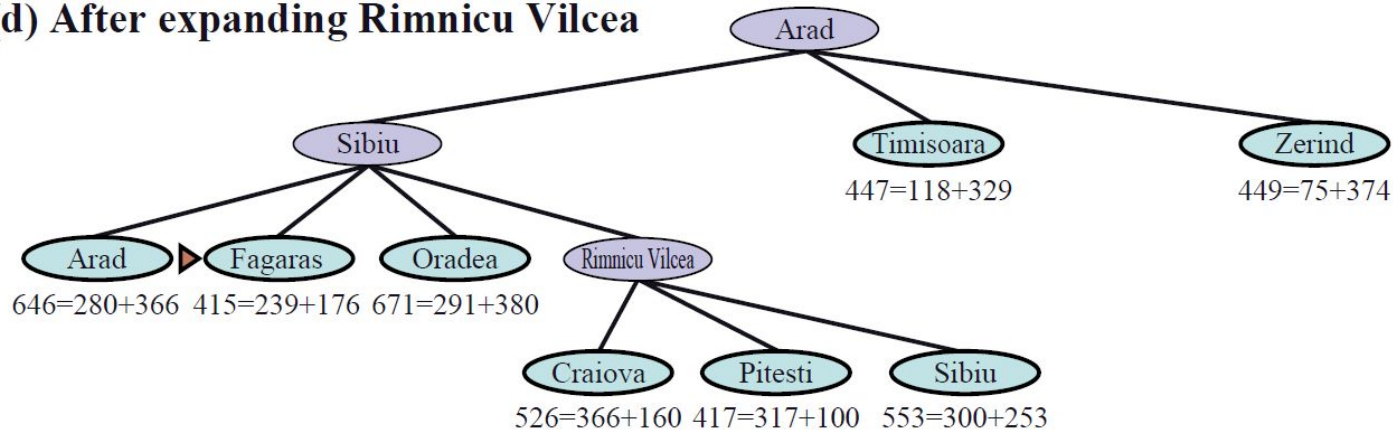
Dist. Linha Reta

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

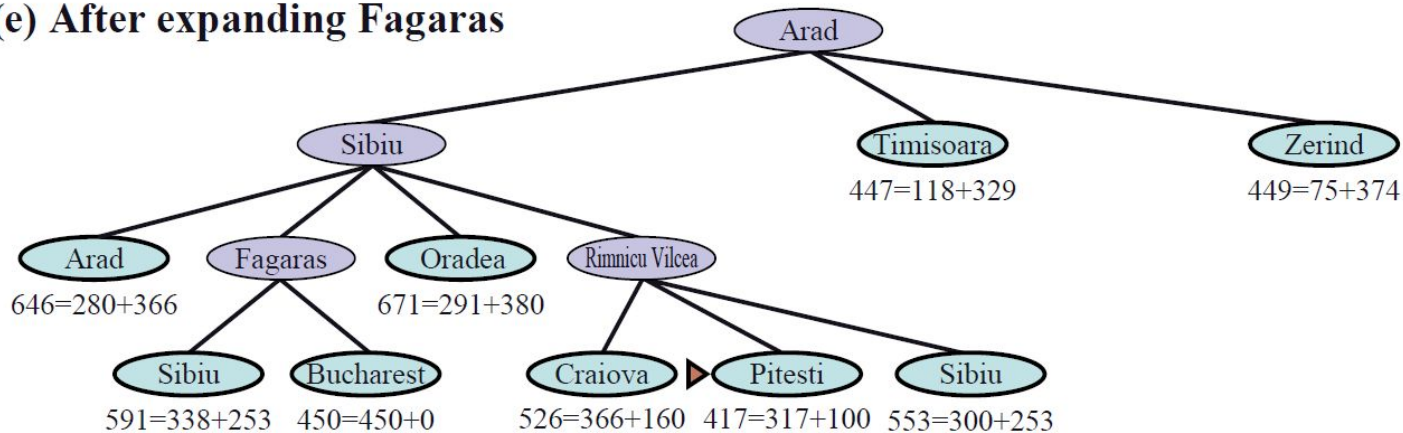
Algoritmo A*

Viagem na Romênia: Arad → Bucureste

(d) After expanding Rimnicu Vilcea



(e) After expanding Fagaras

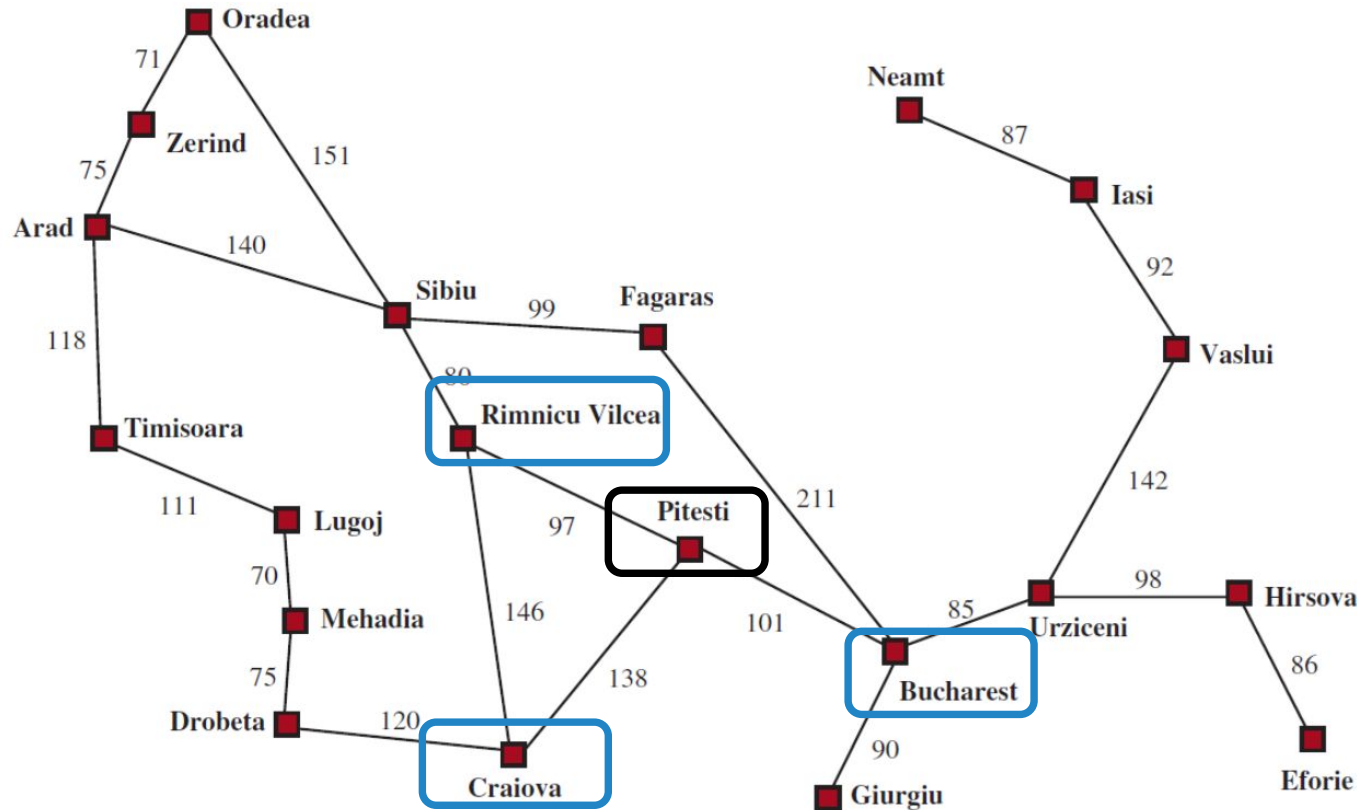


O custo de Bucureste por este caminho é maior do que o aproximado por Pitesti

Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Algoritmo A*

Viagem na Romênia: Arad → Bucareste

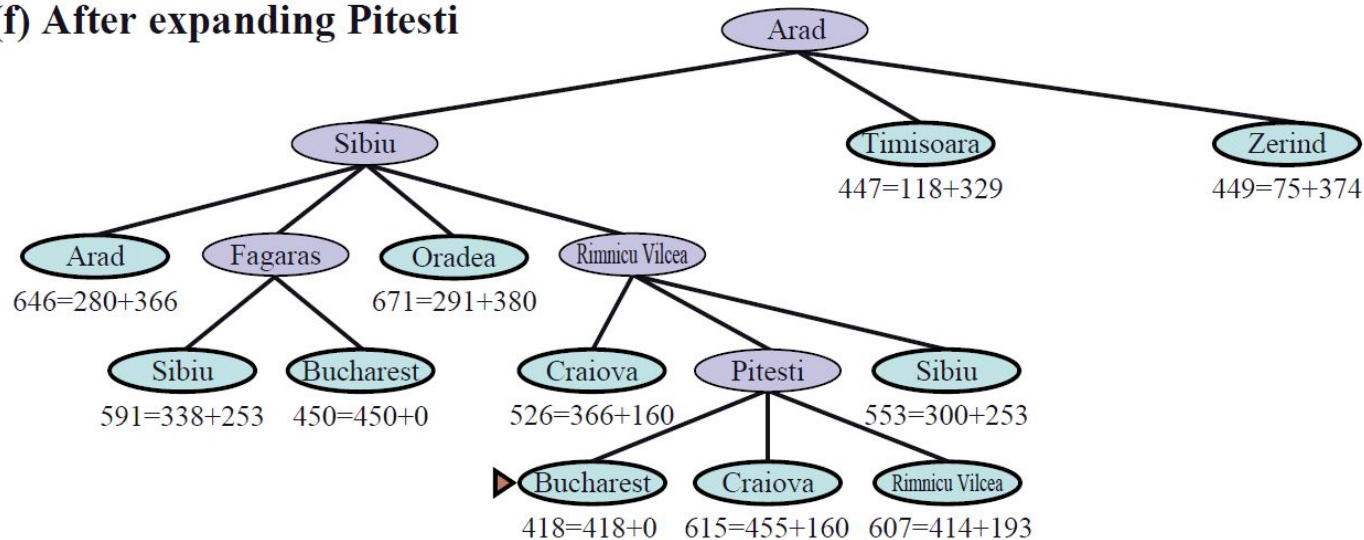


Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Algoritmo A*

Viagem na Romênia: Arad → Bucureste

(f) After expanding Pitesti



Agora sim chegamos!
O custo desse nó é o menor dentre todos os que estão na fronteira

Dist. Linha Reta	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu V.	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Algoritmo A*

Ideia → Expandir o nó de acordo com a função:

$$f(n) = g(n) + h(n)$$

- ▷ $g(n)$: **custo real** de chegar ao nó atual
- ▷ $h(n)$: **estimativa do custo** do nó atual para o objetivo

IMPORTANTE: se $h(n)$ satisfizer algumas condições,
a busca A* será completa e ótima

Algoritmo A*: otimalidade

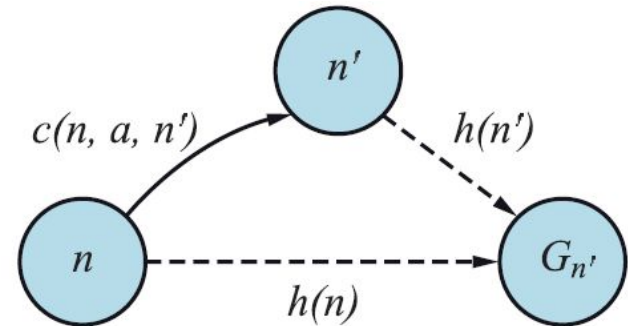
Admissibilidade → Heurística nunca deve superestimar o custo de atingir o objetivo

- ▷ Valor da heurística é menor ou igual ao custo real para chegar ao objetivo
 - **Viagem na Romênia:** distância em linha reta é uma heurística admissível pois o caminho mais curto entre dois pontos quaisquer é uma linha reta
 - Logo, a solução encontrada é **ótima**

Algoritmo A*: otimalidade

Consistência → Heurística é consistente se, para um nó n e os seus sucessores n' gerados por uma ação a , o custo estimado de atingir o objetivo a partir de n não é maior que o custo de chegar a n' somado ao custo estimado de n' para o objetivo

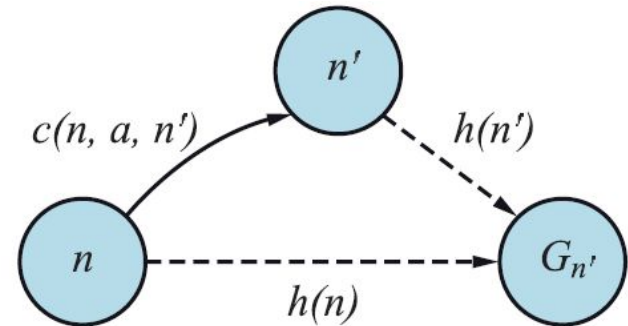
- ▷ $h(n) \leq c(n, a, n') + h(n')$
- ▷ É um tipo de **desigualdade triangular**
- ▷ Consistência implica **admissibilidade**



Algoritmo A*: otimalidade

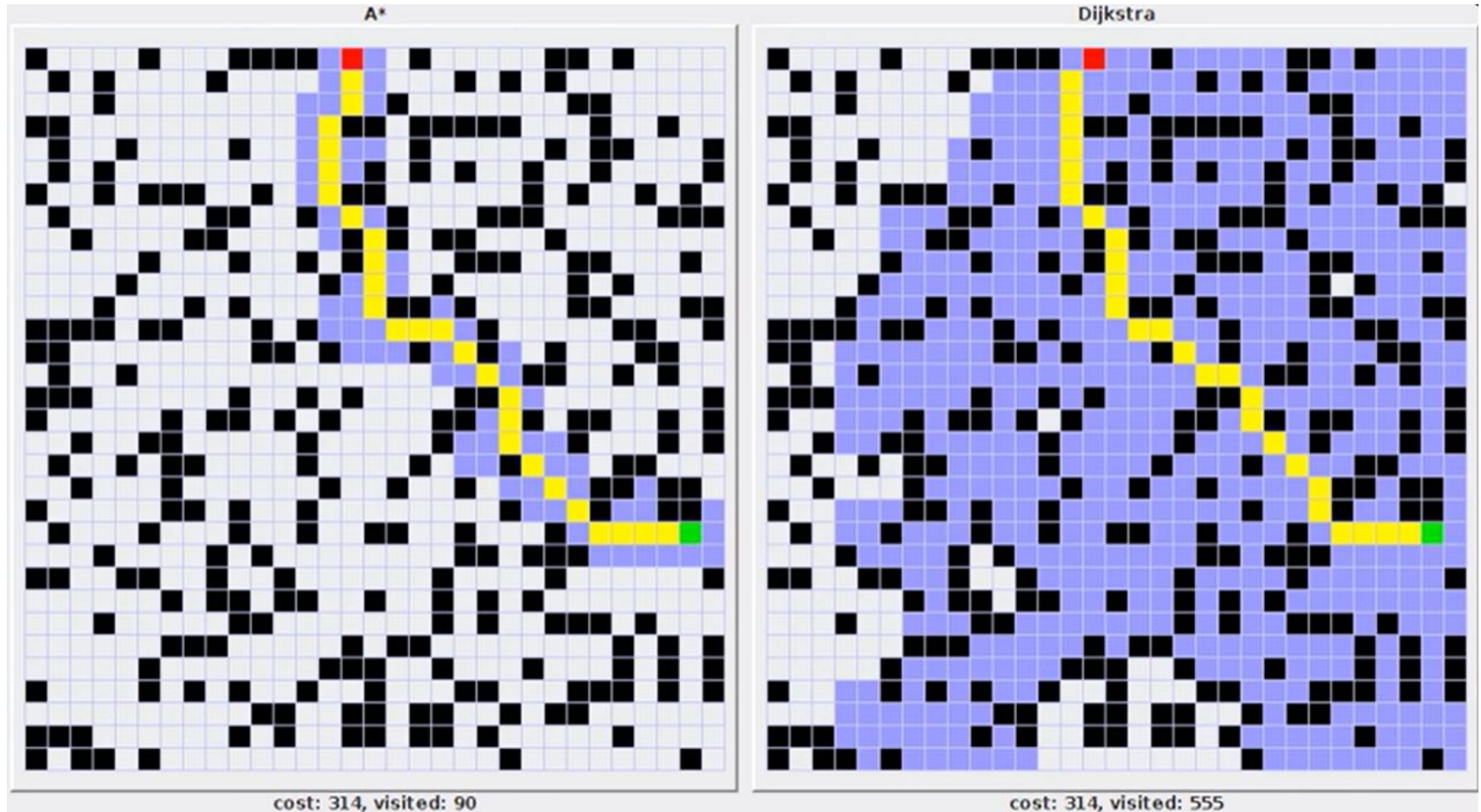
Consistência → Heurística é consistente se, para um nó n e os seus sucessores n' gerados por uma ação a , o custo estimado de atingir o objetivo a partir de n não é maior que o custo de chegar a n' somado ao custo estimado de n' para o objetivo

- ▷ $h(n) \leq c(n, a, n') + h(n')$
- ▷ É um tipo de **desigualdade triangular**
- ▷ Consistência implica **admissibilidade**



IMPORTANTE: Uma heurística consistente garante que toda vez que o nó é expandido, o caminho ótimo até ele foi encontrado

A* vs. Custo Uniforme (*Dijkstra*)



Algoritmo A*

Avaliação do algoritmo

- ▷ **Completo:** sim
- ▷ **Ótimo:** sim (se heurística admissível e consistente)
- ▷ **Complexidade**
 - Tempo: dependendo da heurística, é exponencial
 - Espaço: os nós são mantidos em memória

3.

Funções Heurísticas

Um olhar mais atento sobre a sua natureza

Funções Heurísticas

“Escolher uma heurística adequada é fundamental para o bom funcionamento dos algoritmos de busca com informação.”

- ▷ Como escolher uma boa heurística?
- ▷ É possível criar heurísticas de forma automática?

8-puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- ▷ Custo médio de uma solução é de cerca de **22 passos**
- ▷ O fator de ramificação (*branching factor*) é cerca de **3**
 - Meio → 4 movimentos; Canto → 2 movimentos; Borda → 3 movimentos
 - $3^{22} \approx 3.1 \times 10^{10}$ estados investigados
 - Eliminando estados repetidos: 170.000
 - É factível, mas e o 15 puzzle?

8-puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- ▷ Custo médio de uma solução é de cerca de **22 passos**
- ▷ O fator de ramificação (*branching factor*) é cerca de **3**
 - Meio → 4 movimentos; Canto → 2 movimentos; Borda → 3 movimentos
 - $3^{22} \approx 3.1 \times 10^{10}$ estados investigados
 - Eliminando estados repetidos: 170.000
 - É factível, mas e o 15 puzzle?

Aumenta demais!
Precisamos encontrar
uma boa heurística

Heurísticas para o 8-puzzle

- ▷ h_1 : Número de peças na posição errada
 - Na figura abaixo, $h_1 = 8$
 - **É admissível** porque cada peça será movimentada pelo menos uma vez

- ▷ h_2 : Soma das distâncias das peças de suas posições-alvo
 - Não considera diagonais (*Manhattan Distance*)
 - Na figura, $h_2 = 3+1+2+2+2+3+3+2 = 18$
 - **É admissível** pois todo movimento que pode ser feito move uma peça
 - um passo mais perto do objetivo

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Qualidade da Heurística

Effective Branching Factor (b^*)

- ▷ Seja N o número de nós gerados pelo A^* e d a profundidade da solução
- ▷ b^* é o *branching factor* de uma árvore uniforme com $N+1$ nós
 - $N+1 = 1+b^*+(b^*)^2+...+(b^*)^d$
 - Por exemplo, se o A^* encontra uma solução para o 8-puzzle na profundidade $d=5$ expandindo $N=52$ nós:
 - $b^*=1.92$
- ▷ Uma boa heurística deve apresentar b^* o mais próximo possível de 1

Qualidade da Heurística

d	Search Cost (nodes generated)			Effective Branching Factor		
	BFS	$A^*(h_1)$	$A^*(h_2)$	BFS	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2.01	1.42	1.34
8	368	48	31	1.91	1.40	1.30
10	1033	116	48	1.85	1.43	1.27
12	2672	279	84	1.80	1.45	1.28
14	6783	678	174	1.77	1.47	1.31
16	17270	1683	364	1.74	1.48	1.32
18	41558	4102	751	1.72	1.49	1.34
20	91493	9905	1318	1.69	1.50	1.34
22	175921	22955	2548	1.66	1.50	1.34
24	290082	53039	5733	1.62	1.50	1.36
26	395355	110372	10080	1.58	1.50	1.35
28	463234	202565	22055	1.53	1.49	1.36

Figure 3.26 Comparison of the search costs and effective branching factors for 8-puzzle problems using breadth-first search, A^* with h_1 (misplaced tiles), and A^* with h_2 (Manhattan distance). Data are averaged over 100 puzzles for each solution length d from 6 to 28.

Qualidade da Heurística

d	Search Cost (nodes generated)			Effective Branching Factor		
	BFS	$A^*(h_1)$	$A^*(h_2)$	BFS	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2.01	1.42	1.34
8	368	48			1.40	1.30
10	1033	116			1.43	1.27
12	2672	279			1.45	1.28
14	6783	678			1.47	1.31
16	17270	1683	364	1.74	1.48	1.32
18	41558	4102	751	1.72	1.49	1.34
20	91493	9905	1318	1.69	1.50	1.34
22	175921	22955	2548	1.66	1.50	1.34
24	290082	53039	5733	1.62	1.50	1.36
26	395355	110372	10080	1.58	1.50	1.35
28	463234	202565	22055	1.53	1.49	1.36

Então h_2 vai ser sempre melhor que h_1 ?

Figure 3.26 Comparison of the search costs and effective branching factors for 8-puzzle problems using breadth-first search, A^* with h_1 (misplaced tiles), and A^* with h_2 (Manhattan distance). Data are averaged over 100 puzzles for each solution length d from 6 to 28.

Dominância de Heurísticas

- ▷ Pela definição das heurísticas, podemos dizer que, para qualquer nó n , $h_2(n) \geq h_1(n)$
 - Dizemos então que h_2 **domina** h_1
 - Ou seja, h_2 expande menos nós que h_1
 - Considerando C^* como o custo da solução ótima, todos os nós com $f(n) < C^*$ serão expandidos
- ▷ É vantajoso escolher heurísticas com valores grandes, desde que elas não ultrapassem o custo real e o tempo para computá-las não seja muito grande

Criação de Heurísticas

- ▷ Heurísticas admissíveis podem ser obtidas a partir de soluções ótimas de versões **relaxadas** do problema
 - Problema relaxado é um problema com menos restrições que o original (mais arestas na árvore de busca)
- ▷ O custo de uma solução ótima de uma versão relaxada do problema é uma heurística admissível para o problema original
- ▷ É importante que a solução do problema relaxado seja simples de se obter

Criação de Heurísticas

- ▷ 8-puzzle original
 - Uma peça pode se mover do quadrado A para B se A for adjacente a B e B estiver vazio
- ▷ 8-puzzle relaxado (removendo restrições)
 - Uma peça pode se mover do quadrado A para B $\rightarrow$ derivamos h_1
 - Uma peça pode se mover do quadrado A para B se A for adjacente a B $\rightarrow$ derivamos h_2 (*distância de Manhattan*)
 - Uma peça pode se mover do quadrado A para B se B estiver vazio

Referências

RUSSELL, Stuart J.; NORVIG, Peter. **Artificial intelligence: a modern approach**. 4th ed. Pearson, 2021. (*Capítulo 3*)